COS568 Learned Index
Milestone 1 Report

Liv d'Aliberti

March 29, 2026

Setup

Workflow. I executed the provided `scripts/run_all.sh` pipeline through an exclusive Slurm batch job on the local cluster, so the full scripted workflow ran in a stable CPU environment rather than on a shared login node. That pipeline downloads the datasets, generates all four scripted workloads, builds the benchmark, runs the experiments, and writes the raw outputs to `results/*.csv`. For each workload, the benchmark already repeats each configuration three times (`-r 3`). For B+Tree and Dynamic PGM, I averaged the three throughput values for each hyperparameter row and kept the row with the highest mean throughput. Mixed workloads were also generated successfully, but Milestone 1 focuses on lookup-only throughput, insert throughput, and lookup throughput after insertion.

Local Environment. The handout recommends running on Adroit, loading `anaconda3/2023.3`, and using a per-user `/scratch/network` workspace for experiments. For this submission, I used the same sbatch-based workflow locally instead of Adroit. The repository and outputs for this run were kept under `/n/fs/similarity/kalshi/COS568-LI-SP26`, and the plotting step was executed in a local Python virtual environment after the benchmarks completed.

Compute Environment. The measurements in this report come from Slurm job `27436584`, which completed successfully in 32 minutes and 53 seconds on host `node204`. The job used the `cs` partition with an exclusive allocation of one node, `1` task, `8` CPUs per task, and `64` GB of requested memory. The node configuration reported by Slurm was `x86_64`, 2 sockets, 26 cores per socket, 2 threads per core, 104 logical CPUs total, and about 515 GB of RAM. The batch script also used `--hint=nomultithread`, `srun --cpu-bind=cores`, and pinned the runtime environment with `OMP_NUM_THREADS=1`, `OMP_PROC_BIND=true`, and `OMP_PLACES=cores`. The benchmark itself ran with `1` application thread, so the main benefit of the Slurm setup was isolating the run from outside contention and keeping CPU placement stable.

Experimental Setup. The report covers the three provided 100M-key public `uint64` datasets: Facebook, Books, and OSMC. The scripted pipeline generates all four workloads per dataset: (1) lookup-only with 2M operations and negative lookup ratio 0.5, (2) insert-lookup with 2M operations, insert ratio 0.5, and negative lookup ratio 0.5, (3) mixed insert-lookup with insert ratio 0.9, and (4) mixed insert-lookup with insert ratio 0.1. Milestone 1, however, only asks for the lookup and insertion comparison, so the quantitative analysis in this report uses the first two workloads: the lookup-only workload and the insert-lookup workload with the scripted phase order of 1M inserts followed by 1M lookups. The benchmark output is repeated three times per row, and the report averages those three values. The result files use the normalized headers `lookup_throughput_mops` and `insert_throughput_mops`; these correspond to the assignment's read-throughput and insert-throughput metrics. For B+Tree and Dynamic PGM, I select the hyperparameter row with the highest mean throughput for the relevant metric, while LIPP has no comparable hyperparameter sweep in these scripts.

Results

Selected Best Variants. In this sbatch run, the best B+Tree configuration was consistently `LinearSearch` with node size 8. Dynamic PGM was more workload-sensitive: lookup-only favored a small error bound ($\epsilon = 16$), while insert-heavy workloads preferred much larger error bounds, ranging from 128 to 1024, with the best search method varying by dataset.

Lookup and Insert Results. The main results are summarized in Table 1 and Figure 1. Across all three datasets, LIPP is overwhelmingly stronger on both lookup-only throughput and lookup throughput after insertion, while Dynamic PGM is clearly the strongest on pure insertion throughput. B+Tree is stable but slower than the learned alternatives on every Milestone 1 metric in this run.

	Lookup-Only			Insert Throughput			Lookup After Insertion		
Dataset	B+Tree	DPGM	LIPP	B+Tree	DPGM	LIPP	B+Tree	DPGM	LIPP
Facebook	1.967	2.579	419.264	1.220	7.750	3.072	1.793	0.616	432.325
Books	1.947	2.682	426.688	1.219	7.525	3.867	1.795	0.667	437.802
OSMC	2.425	2.585	424.154	1.215	6.688	2.339	2.243	0.613	433.212

Table 1: Average throughput in Mops/s, using the best mean-performing hyperparameter row per index and workload from the exclusive sbatch run. The table shows the central Milestone 1 pattern: LIPP dominates read-heavy metrics, Dynamic PGM dominates insertion throughput, and B+Tree remains consistently below both learned indexes.

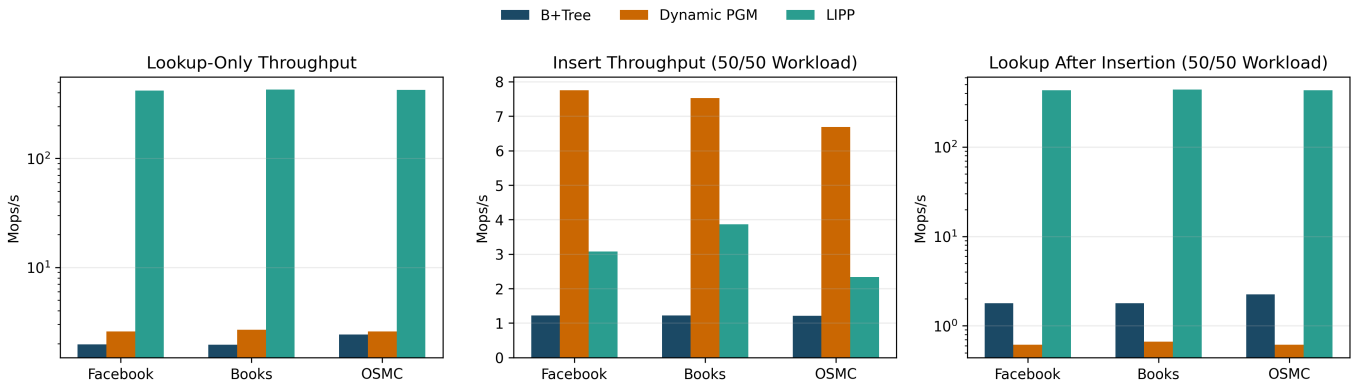

Figure 1: Milestone 1 metrics from the sbatch run. The figure covers the lookup-only workload and the 50/50 insert-lookup workload on Facebook, Books, and OSMC. The lookup-oriented plots use a log scale because LIPP is orders of magnitude faster on reads in this run, while the insert panel makes Dynamic PGM's update advantage immediately visible.

Analysis

Comparison and Explanation. Three trends are consistent across all datasets.

- **Lookup-only: LIPP is the clear winner, and Dynamic PGM beats B+Tree.** LIPP achieves roughly 419 to 427 Mops/s, while Dynamic PGM reaches 2.58 to 2.68 and B+Tree reaches 1.95 to 2.43. LIPP's learned model predicts precise slots, so it avoids both the pointer chasing of B+Tree and the bounded local correction search used by Dynamic PGM.
- **Insertion: Dynamic PGM is the clear winner.** Dynamic PGM reaches 6.69 to 7.75 Mops/s, while LIPP is around 2.34 to 3.87 and B+Tree is around 1.21 to 1.22. This matches the intended update behavior of Dynamic PGM: insertions are amortized through its dynamic learned structure, while B+Tree pays for splits and LIPP pays for conflict-driven restructuring and a much heavier learned layout.
- **Lookup after insertion: LIPP still dominates read performance.** After the 50/50 insert-lookup workload, LIPP still achieves about 432 to 438 Mops/s on reads, far above B+Tree (1.79 to 2.24) and Dynamic PGM (0.61 to 0.67). Once keys are incorporated, LIPP keeps its exact-position lookup behavior, whereas Dynamic PGM still performs approximate prediction followed by local search.

Takeaway. The sbatch-based run reinforces the intended tradeoff behind the project. LIPP is the best choice when the workload is read-heavy, Dynamic PGM is the best choice when insertion throughput matters, and B+Tree remains a solid but slower baseline that does not win any Milestone 1 metric in this experiment.

Appendix: Mixed Workloads

The provided pipeline also generates the two mixed insert-lookup workloads with insert ratios 0.1 and 0.9. These are outside the core Milestone 1 comparison, but I include them here to document that the full scripted experiment ran successfully in the sbatch environment and to show the throughput behavior under hybrid read/write pressure.

Mixed Workload with 10% Inserts. When the workload is mostly lookups with a small fraction of inserts, LIPP is again the strongest index by a large margin. Its mixed throughput reaches about 17.9 to 27.1 Mops/s across the three datasets, while B+Tree remains around 1.7 to 2.0 and Dynamic PGM stays around 1.06 to 1.19. This is consistent with the lookup-heavy setting: even though updates are interleaved, LIPP still benefits from extremely fast read-side behavior.

Mixed Workload with 90% Inserts. When inserts dominate the mixed workload, the outcome shifts toward Dynamic PGM. It has the best mixed throughput on Facebook and OSMC at about 3.50 and 3.32 Mops/s, while LIPP is slightly better on Books at 4.35 Mops/s. B+Tree remains the slowest option at roughly 1.22 to 1.23 Mops/s. This mixed case is useful because it shows that the same read-vs-write tradeoff seen in the main milestone results also appears when operations are interleaved instead of phase-separated.

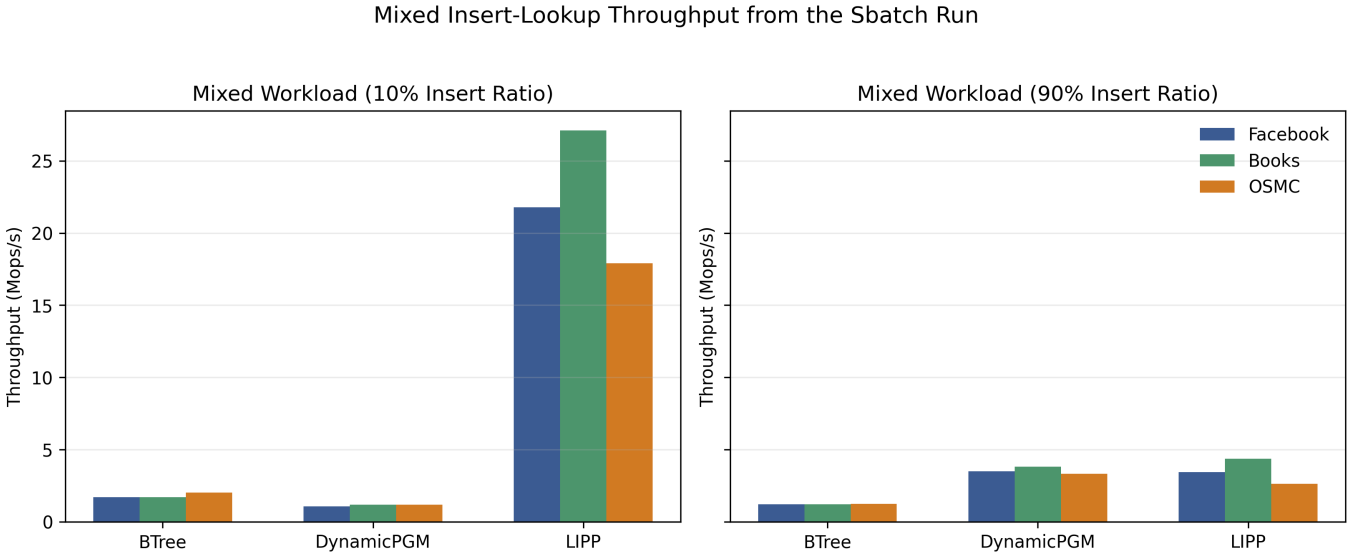

Figure 2: Average mixed-workload throughput in Mops/s from the sbatch run, using the best mean-performing hyperparameter row per index and workload. The left panel shows the lookup-heavy mixed workload (10% inserts), where LIPP clearly dominates. The right panel shows the insert-heavy mixed workload (90% inserts), where Dynamic PGM is usually strongest, though Books is a close exception in this run.